

최금영

Backend Engineer · Kotlin / Java · Spring Boot · 10년차 · MSA 전환 · 이벤트 기반 · AI-native 개발 프로세스

GitHub github.com/geumyeongchoi 기술 블로그 Notion 경력 소개

레거시 주문 시스템을 서비스 중단 없이 MSA + 카드결제로 전환하고, AI 코딩 에이전트가 안전하게 기여할 수 있는 개발 하네스(규칙 → 사전 차단 혹은 검증 게이트 → 티켓 → 스펙 → 구현 → 인수테스트)를 설계·운영하는 백엔드 개발자입니다. 식자재·외식 커머스, 제조 AI 품질검사, AIOps, CRM/보안 솔루션 도메인을 거쳤습니다.

이 페이지는 2026년 9월, 2주 동안 진행한 사이드 프로젝트 3건의 기록입니다. 각 프로젝트는 문제 정의 → 설계 판단(대안 비교) → 실측 수치 → 배운 것 순서로 정리했고, 모든 수치는 저장소 안의 결과 파일에서 인용했습니다(추정치 없음).

개요 — 왜 이 세 가지인가

채용 공고에서 반복되는 요구(RAG/LLM 연동 · 대용량 트래픽과 고가용성 · Kubernetes/CI-CD/모니터링)를 각각 하나의 작은 프로젝트로 실증하되, 서로 연결되도록 설계했습니다. 1·2번 서비스는 3번 플랫폼 위에 배포됩니다.

프로젝트	한 줄 정의	핵심 실측
docmind	문서를 올리면 그 문서만 근거로 답하고 출처를 붙이는 사무용 RAG 챗봇. 사내망(Ollama) ↔ 외부 모델 (OpenAI/Anthropic) 프로파일 전환.	정답 포함률 43% → 76.7% (프롬프트 회귀 평가) · 하이브리드 검색 +6.7pt · 근거 없으면 LLM 미호출
flashgate	선착순 한정수량 주문 API. Redis Lua 원자 예약 + 아웃박스/Kafka 확정, Sentinel failover와 인스턴스 강제 종료로 카오스 테스트로 검증.	동시 1,000 요청 초과판매 0 · spike 1,496 rps p50 1.6 ms · Redis master kill 중 500 오류 0 · 앱 SIGKILL 실패율 0.10%
homelab-platform	자체 Linux 서버 k3s 위에 두 서비스를 HTTPS 도메인으로 배포하고, PR 머지 한 번으로 롤아웃, Grafana 한 화면에서 로그·메트릭·트레이스.	Helm 차트 2종 · GitHub Actions → GHCR → 롤링 배포 · LGTM 스택 (서버 배포 후 수치 갱신 예정)

1 · docmind — 사내 문서 RAG 챗봇 (private / public 전환형)

Kotlin 2.2 · Spring Boot 3.5 · Spring AI 1.1 · PostgreSQL 17 + pgvector · Ollama / OpenAI / Anthropic · MCP Server · github.com/geumyeongchoi/docmind

문제

사규·기술 문서·회의록은 흩어져 있고 키워드 검색은 "연차 이월돼요?" 같은 질문에 답하지 못한다. 문서를 외부 LLM에 보내면 안 되는 조직이 많고, 반대로 정확도가 중요할 때는 상용 모델을 쓰고 싶다.

RAG 하이브리드 검색(RRF) SSE 스트리밍 프로파일 전환 평가셋 회귀 MCP

아키텍처

```
[Web UI] —SSE→ [docmind-api Kotlin · Spring Boot 3.5 · Spring AI 1.1]
├─ Ingest   : Tika → OverlappingTokenSplitter(600/80) → Embedding → pgvector
├─ Retrieval: 벡터 || 전문검색(tsvector) → RRF 병합 → topK 6
├─ Chat     : 근거 없으면 LLM 미호출 / ChatClient + MessageChatMemory / 질문 세이프가드
├─ Viewer   : 출처 카드·[출처: n] → 원문 뷰어(체크 강조, PDF #page)
└─ MCP      : search_docs · ask_docs (Claude Desktop / Claude Code 에서 도구로 사용)
profile=private → Ollama (bge-m3 1024d · gemma3:4b)           데이터가 서버 밖으로 안 나감
profile=public  → OpenAI (text-embedding-3-small 1536d · gpt-4.1-mini)
profile=claude  → Anthropic(chat) + Ollama bge-m3(embedding)  질문+검색 체크만 외부로
PostgreSQL 17 + pgvector (documents · document_files · docmind_private · docmind_public · tsvector GIN)
```

설계 판단 (대안 비교)

- **Spring AI 단일 JVM 서비스 vs Python LangChain 별도 서비스** — 배포·관측·테스트를 한 벌로. Loader/Splitter/VectorStore/Advisor가 LangChain 개념과 1:1이라 학습 비용 없이 같은 설계를 옮길 수 있었다.
- **pgvector + tsvector 하이브리드(RRF) vs 벡터 단독/전용 벡터DB** — 추가 인프라 0, 고유명사·코드명 recall 보완. 파라미터가 k 하나뿐이라 튜닝이 단순하다.
- **근거 없으면 LLM을 호출하지 않는다 vs 프롬프트로 억제** — 비용 0, 환각 차단, 지연 최소. 임계값은 평가셋으로 정한다.
- **프로파일별 벡터 테이블 분리** — 임베딩 차원(1024 vs 1536)이 다르면 같은 인덱스에 섞을 수 없다. Anthropic 은 임베딩 API가 없어 로컬 임베딩과 조합했고, 덕분에 private 로 올린 문서를 재인제스트 없이 공유한다.
- **세이프가드를 질문에만 적용** — 프롬프트 전체를 검사하는 SafeGuardAdvisor는 인젝션 문구가 든 문서가 검색되는 순간 정상 질문까지 거절했다.

실측 — 평가셋 30문항, 같은 문서·같은 모델(GEMMA3:4B)

시스템 프롬프트	검색	정답 포함률	p50
v1 엄격 거절 우선	hybrid	43.3%	4.6s
v2 "관련 문장 있으면 반드시 답"	hybrid	70.0%	5.5s
v3 번호 인용 + 거절 재강조	hybrid	56.7%	5.4s
v3 동일	vector only	50.0%	5.1s
v4 채택	hybrid	76.7%	5.9s

체크 크기/오버랩	체크 수	정답 포함률	p50
300 / 37	17	83.3%	4.7s
600 / 80 (기본)	9	76.7%	5.9s
1000 / 125	8	80.0%	7.0s

환경: MacBook Pro(Apple Silicon) · profile=private · bge-m3 · topK 6 · 문서 8종. 원본: eval/results/*.md

배운 것

- **프롬프트 문구 하나로 43% ↔ 77%**. 4B 모델은 거절 규칙의 위치와 강도에 매우 민감하다. 프롬프트는 코드처럼 평가셋으로 회귀 테스트해야 한다는 것이 이 표의 결론이고, 그래서 ./gradlew eval 한 번으로 표가 갱신되게 만들었다.
- **하이브리드 vs 벡터 단독 +6.7pt**. 차이를 만든 것은 고유명사 문항(PRC_S00R0U001, E4012, 토픽명)이었다.
- **인용 정확률 100%는 아직 의미가 없다**. 코퍼스가 9체크라 topK 6이 거의 전부를 덮는다. 문서 30+로 늘려 재측정하는 것이 다음 순서.
- qwen3:4b 는 기본이 thinking 모드라 RAG 컨텍스트에서 응답 103초 → gemma3:4b 로 교체. Docker Engine 29 는 Testcontainers 구버전 API를 거절해 DOCKER_API_VERSION=1.44 필요.

[README \(실측 표\)](#) [설계 문서 \(C4 · 시퀀스 · ERD · ADR 8건\)](#) [FigJam 설계도](#) [MCP 연동](#)

2 · flashgate — 선착순 한정수량 주문 API (초저지연 · 고가용성)

Kotlin 2.2 · Spring Boot 3.5 · JDK 21 가상 스레드 · Redis 7 Sentinel(Lettuce) · Kafka · MySQL 8.4 · Resilience4j · Traefik · k6
· github.com/geumyeongchoi/flashgate

문제

재고 100개짜리 상품에 1초에 수천 명이 몰려도 초과판매 0건, p99 수십 ms, 그리고 앱 인스턴스나 Redis 마스터가 죽어도 요청이 "조용히 실패"하지 않아야 한다. 흔한 해법(DB 비관락, 분산락)은 정확하지만 락 대기 때문에 지연이 트래픽에 비례해 늘어난다.

Redis Lua 원자 연산

먹통키

Transactional Outbox

Kafka

Sentinel failover

Resilience4j

k6 카오스

아키텍처

```
client → Traefik(헬스체크 1s · retry 1회) → [flashgate-api x2] JDK 21 가상 스레드
| POST /api/orders {productId, userId, idempotencyKey}
| EVALSHA reserve.lua (재고 확인 + 차감 + 유저 중복 체크 + 예약 기록 = 1 round-trip)
| [Redis Sentinel: master + replica×2 + sentinel×3]
| 성공 시 outbox INSERT(같은 트랜잭션) → 202 {orderId, RESERVED}
|
| [Kafka] order.reserved ← outbox relay(100ms · SKIP LOCKED)
|
| [order-confirm consumer] 모의 PG(5% 거절) → MySQL CONFIRMED → order.confirmed
| 실패 시 compensate.lua 로 재고 복원 + order.cancelled
매 1분 대사: Redis 잔여 + MySQL 확정 + 취소 = 초기 재고 (불일치 → 메트릭 알람)
```

0

초과판매 (동시 1,000 요청 ·
재고 100 → 202 정확히 100
건)

1,496 rps

spike 재고 100 · 5,000명 10s
· 5xx 0

1.6 ms

spike p50 (p99 27.6 ms)

500 오류 0

Redis master kill 중 — 실패는
전부 503 + Retry-After

실측 — 시나리오별 (로컬 DOCKER · 앱 2대 + SENTINEL 3 + TRAEFIK)

시나리오	요청	5xx	p50	p99
spike — 재고 100, 5,000명 10s	15,149	0	1.6 ms	27.6 ms
ramp — 100 → 1,000 rps 55s	39,001	0	2.0 ms	27.3 ms
chaos — Redis master kill 중 300 rps	18,001	5.7% (전부 503)	1.9 ms	203 ms
chaos — 앱 1대 SIGKILL 중 300 rps	18,002	0.10%	2.0 ms	35 ms

Redis master kill: 1차(down-after 5s, 예외 매핑 전) 500 × 2,433(13.5%) → 2차 503 × 1,021(5.7%), 실패 창 ≈ 3.4s. 앱 SIGKILL: Traefik retry 적용 전 1.5% → 0.10%. 원본: k6/results/*.md

설계 판단 — 대안을 구현해서 같은 부하로 비교

전략	5xx	p50	p95	p99
lua Redis Lua 원자 차감 (채택) · Sentinel 경우	0	2.39	23.98	42.65
lua · Redis 직접 연결	0.003%	0.88	1.97	4.32
dblock · MySQL SELECT ... FOR UPDATE	0	1.58	5.40	13.18
redisson · RLock(tryLock 50ms)	50.0%	2.14	8.85	18.14

ramp 100 → 1,000 rps · 재고 10만 · 단위 ms · 세 전략 모두 초과판매 0.

배운 것

- "Lua가 락보다 빠르다"는 이 규모(700 rps, 단일 상품)에서는 자명하지 않았다. 로컬 MySQL 행 락은 충분히 빨랐다. Lua의 진짜 이점은 DB 커넥션을 핫패스에서 완전히 빼는 것(DB 장애·풀 고갈과 분리)과 경합이 커질 때 락 대기가 없다

는 점이며, 더 높은 rps와 원격 DB에서 증명해야 한다.

- **분산락은 경합이 실패율로 전이된다.** tryLock 대기를 늘리면 실패율은 내려가지만 지연이 그만큼 큐잉된다 — 선착순 트래픽에는 구조적으로 불리.
- **가장 큰 변수는 알고리즘이 아니라 연결 경로였다.** Sentinel 경유 Lettuce 가 p95 24 ms, 직접 연결이 2 ms. 원인 후보 (토폴로지 리프레시, ReadFrom, Docker 네트워크 흡)를 다음 단계로 남겨 두었다.
- **failover 창 의 실패는 "빠른 503"으로 바꾸는 것이 목표**이지 0으로 만드는 것이 아니다. Retry(50→100→200ms)는 수 초의 failover를 덮을 수 없으므로 남는 요청은 503 + Retry-After 로 즉시 돌려보내 큐잉·지연 폭증을 막고, 클라이언트는 먹등기로 안전하게 재시도한다. 예외 매핑이 빠지면 같은 장애가 500으로 보인다.

[README \(실측 표 · 대안 비교\)](#) [k6 시나리오 · 결과](#) [reserve.lua](#) · [compensate.lua](#)

3 · homelab-platform — 자체 서버 k3s · CI/CD · 관측 · HTTPS

k3s · cert-manager(Let's Encrypt) · Helm · GitHub Actions → GHCR · kube-prometheus-stack · Loki · Tempo · OpenTelemetry
· github.com/geumyeongchoi/homelab-platform

문제

위 두 서비스를 "내 서버에서 도메인으로 접속 가능한 상태"로 두고, PR 머지 한 번으로 배포되며, 배포 중에도 에러율 0을 유지하고, 문제가 생기면 Grafana 한 화면에서 로그·메트릭·트레이스를 따라갈 수 있어야 한다.

[Kubernetes\(k3s\)](#) [Helm](#) [GitHub Actions](#) [HPA · PDB · probe](#) [OTel 자동계측](#) [LGTM](#)

```
GitHub (docmind / flashgate PR 머지)
| Actions: test → jib 이미지 → GHCR push → kubectl set image (실패 시 rollout undo)
▼
[Linux 서버] k3s (single node · Traefik ingress 내장)
├─ cert-manager + Let's Encrypt → *.도메인 HTTPS
├─ ns docmind      : Deployment(1) + pgvector StatefulSet
├─ ns flashgate    : Deployment(2 · PDB minAvailable=1 · HPA cpu 70% · preStop) + redis(sentinel) + kafka + mysql
├─ ns observability : kube-prometheus-stack · Loki(+Alloy) · Tempo · OTel Collector ← OTel Java agent(initContainer 주입)
└─ ns platform     : sealed-secrets · whoami
```

설계 판단

- **k3s vs Compose / kubeadm / EKS** — Compose는 K8s 운영 경험이 되지 않고, kubeadm은 설치가 하루, EKS는 월 \$70+. k3s는 10분 설치·Traefik 내장·512MB.
- **GHCR + kubectl set image(push형) vs ArgoCD** — 2주 안에는 push형이 현실적. GitOps 전환은 다음 단계로 명시.
- **OTel Java agent 자동계측** — 코드 변경 없이 HTTP/JDBC/Kafka/Redis 스펠. 앱은 OTLP 엔드포인트만 안다.
- **Loki + Tempo(LGTM) vs ELK** — 메모리 1/4, trace_id 클릭 → 로그로 3신호 상관 조회.
- **Sealed Secrets** — 저장소를 공개해도 안전 + 설치 1분.

상태

k3s 설치·점검 스크립트, ClusterIssuer, Actions 파이프라인, Helm 차트 2종(HPA/PDB/probe/OTel 주입), LGTM values, RED 대시보드 JSON, 런북까지 저장소에 준비되어 있고 **서버 배포와 롤링 배포 중 k6 에러율 측정은 진행 중**입니다. 이 섹션의 수치는 배포 후 갱신합니다.

[README](#) [deploy.yml](#) [Helm 차트](#) [런북](#)

개발 방식 — AI 에이전트와 함께, 검증은 게이트로

세 저장소 모두 같은 규칙으로 만들었습니다. 현업에서 설계·운영 중인 "AI 코딩 에이전트 하네스"를 개인 프로젝트에 그대로 적용한 것입니다.

- **규칙은 한 문서에** — 각 저장소의 CLAUDE.md(=AGENTS.md)에 스택, 패키지 구조, HARD-GATE(예: "재고를 바꾸는 코드는 오직 Lua 안에서만", "성능 수치는 k6 결과 파일에서만 인용", "API 키 하드코딩 금지")를 명시. 에이전트가 무엇을 하든 이 규칙을 먼저 읽는다.
- **스펙 먼저** — specs/dayN.md에 DoD와 테스트 시나리오를 쓰고, 그것을 테스트 코드로 옮긴 뒤 구현. 아키텍처 규칙(domain 은 프레임워크 무의존, 유스케이스는 포트만 의존)은 ArchUnit 으로 강제.
- **2단 검증 게이트** — verify.sh --fast(compile + ktlint + 단위/아키텍처, 수십 초) 통과 전 커밋 금지, --full(Testcontainers 통합)은 머지 전. 무거운 게이트는 우회를 낳는다는 경험에서 나온 구성.
- **수치는 재현 가능하게** — ./gradlew eval, k6 run 한 번으로 README 표가 다시 만들어진다. 이 페이지의 모든 숫자는 그 결과 파일에서 가져왔다.
- **역할 분리** — 에이전트(Claude)는 스펙 기준으로 구현·테스트를 수행하고, 설계 판단·검증·커밋은 사람이 확인한다.

기술 스택

Language	Kotlin, Java 21, Python(Airflow)
Framework	Spring Boot 3.x, Spring AI, Spring WebFlux/R2DBC, Spring Batch, Spring Cloud AWS
Data	PostgreSQL(pgvector), MySQL(파티셔닝), Redis(Sentinel/Lua/Redisson), MongoDB, Elasticsearch
Access	jOOQ, JPA/QueryDSL, MyBatis, Spring JDBC
Messaging	Kafka(AWS MSK), AWS SQS FIFO, Transactional Outbox
Infra	AWS(S3·SQS·MSK·AppConfig·Secrets Manager), Docker, Kubernetes(k3s), Helm, GitHub Actions, Traefik, Testcontainers, Jib
Observability	Micrometer, OpenTelemetry, Prometheus/Grafana, Loki, Tempo
Quality	Kotest, JUnit5, MockK, Cucumber(BDD), ArchUnit, ktlint, k6, Resilience4j
AI-native	Claude Code(hooks/skills/agents), Codex/AGENTS.md, Cursor, MCP(서버 구현·클라이언트 연동), 스펙 주도 개발